

MICROSERVICES SERIES

10 Design Patterns You Must Know

Netflix • Amazon • Uber • Kubernetes

Swipe to learn → 🖱️

Every great system you use daily — Google Maps, Instagram, Netflix — is powered by microservices. But microservices without design patterns is just organised chaos. These 10 patterns are the real secret behind systems that scale to millions.



All 10 Patterns

① API Gateway

② DB per Service

③ Circuit Breaker

④ Svc Discovery

⑤ Event Sourcing

⑥ CQRS

⑦ Saga

⑧ Strangler Fig

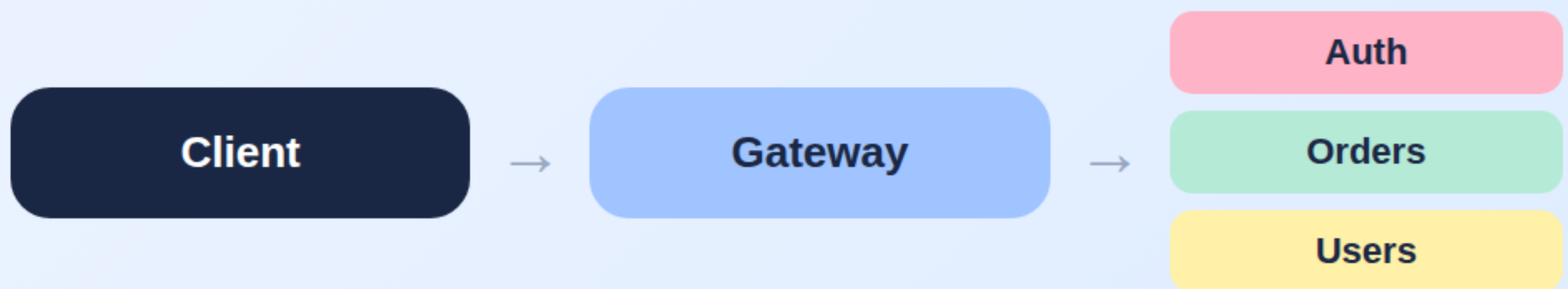
⑨ Bulkhead

⑩ Sidecar

Think of these 10 patterns as your architect's toolkit. Before you write a single line of microservice code, knowing which pattern solves which problem will save you weeks of pain, rewriting, and production fires.

1 API Gateway

🚪 Single entry point



✓ Routes • Auth • Load Balances • Rate Limits

Used by Netflix for **1B+ daily requests**

Imagine 10 microservices all exposed directly to the internet — every client needs to know every service address, and security is a mess. API Gateway is the single smart door that handles routing, auth, and rate limiting so your services stay clean and protected.

2

Database per Service



Own your data



Orders

PostgreSQL



Users

MongoDB



Inventory

Redis

Each service owns its data. No shared DB chaos. 🔒

When two services share a database, a bad migration in one breaks the other. It's like two people editing the same file at once. Database per Service gives each service full ownership of its data — so they can scale, fail, and evolve completely independently.



3

Circuit Breaker

⚡ Stop the chain reaction

Service
A ✓



Service
B ✗



Service
C ✓

🔌 B fails → circuit opens → A & C stay alive

CLOSED ✓
Normal flow

OPEN ✗
Block calls

HALF ⚠
Test recovery

Picture a traffic jam — one blocked lane slows everyone. In microservices, one slow service can freeze the entire system. Circuit Breaker watches for failures, cuts the connection instantly, and lets everything else keep moving — just like a real electrical breaker.

4

Service Discovery

 GPS for services



Service
Registry

● Auth — 3 instances

● Payment — Scaling

● Orders — 5 instances

Uber manages 1000s of services dynamically 🚗

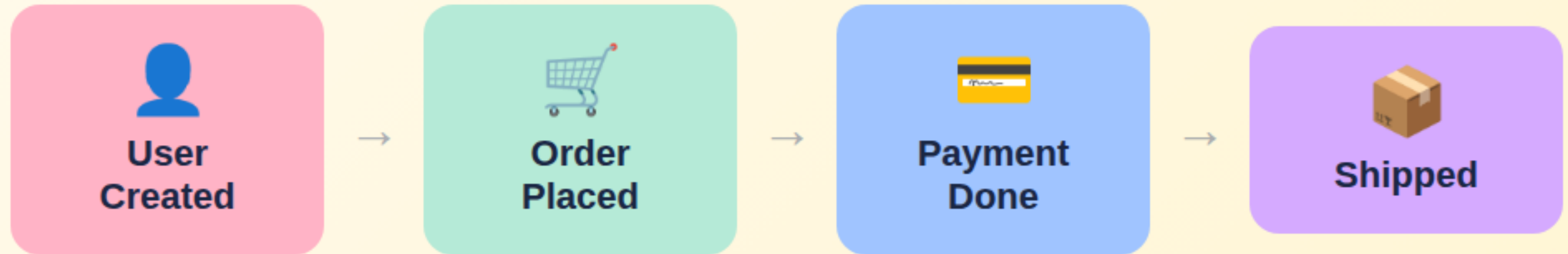
In a cloud environment, services start and stop constantly. How does Service A find Service B if its IP changes every few minutes? Service Discovery acts like a live phone book — services register themselves and find each other automatically, no hardcoding needed.

5

Event Sourcing



Store every change



Banks use this for complete audit trails

Something breaks? Replay events. Find truth. ✓

Most databases only remember the latest state — like overwriting a document. Event Sourcing saves every single change as an event. So you can rewind time, replay history, find exactly what went wrong, and rebuild state from scratch — perfect for banking and audit systems.



6

CQRS



Command Query Responsibility Segregation



WRITE

Commands
Safe & Secure

≠



READ

Queries
Fast & Optimized

LinkedIn: millions of profile reads daily 🧑💼

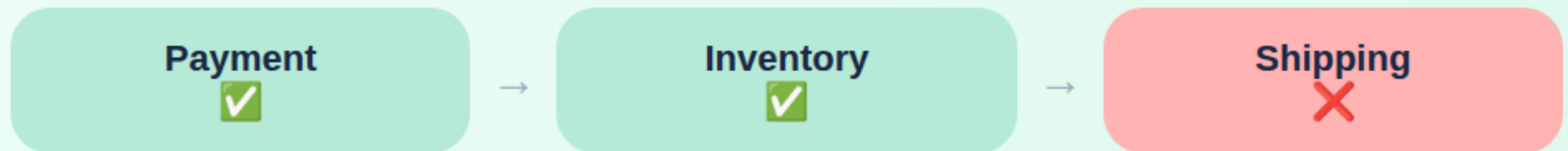
Reading a list of products and writing a new order are completely different operations. Why force them to share the same model? CQRS splits reading and writing apart — reads become blazing fast, writes stay consistent, and each side scales on its own.

7

Saga Pattern

 Distributed transactions

You order on Amazon...



 Saga auto-rolls back Payment + Inventory

No mess. No lost money. No angry customers 😊

You book a flight: payment goes through ✓, seat reserved ✓, but email service crashes ✗. Without Saga, your money is gone but booking is incomplete. Saga coordinates each step and automatically undoes previous ones if anything fails — so transactions always finish cleanly.

8

Strangler Fig

 Migrate safely



Old Monolith



 Auth Service

 Orders Service

 Payment

How Amazon migrated — piece by piece, zero downtime 🏆

You have a 5-year-old monolith. Rewriting it overnight is too risky. Strangler Fig lets you carve out one feature at a time into a microservice, while the old system keeps running. Slowly, the monolith shrinks and the new system grows — zero downtime, zero drama.

9

Bulkhead



Isolate failures like ship compartments



Search Pool

10 threads



Payment Pool

OVERLOADED ❌



Orders Pool

10 threads

Payment fails? Search & Orders keep running! ✓

Imagine a ship where one flooded compartment sinks the whole vessel. Bulkhead pattern puts walls between services — each gets its own thread pool and resources. If payments get flooded with traffic, search and orders keep sailing smoothly, completely unaffected.



10

Sidecar

 Your service's silent partner



Your Service

+



Logging



Security



Monitor

Kubernetes uses Envoy proxy sidecars 

Which pattern do you use? Comment  Follow for more 

Every service needs logging, security monitoring, and tracing. But writing that code in every service is repetitive and messy. Sidecar attaches a helper container to your service automatically — it handles all the infrastructure work silently, so your service stays focused on what it does best.